

Projet QAOps

Automatisation des tests et assurance qualité d'une application web

Rapport final synthétique

Université Hassan 2 — Faculté des Sciences Aïn Chock

4 octobre 2026

Selenium

Newman

JMeter

OWASP ZAP

Jenkins

1. Contexte, objectifs et méthode

Le projet place l'équipe dans le rôle de QA d'une application web de gestion d'étudiants. Les cibles pédagogiques imposées sont Formy Project pour l'interface et Reqres pour l'API REST. L'objectif est d'automatiser la validation fonctionnelle, de mesurer un comportement sous charge, d'identifier des risques de sécurité, puis de rendre l'ensemble reproductible par intégration continue.

Objectifs réalisés

- 10 scénarios tracés : 5 UI et 5 API.
- Tests UI Selenium exécutés en mode Chrome headless avec rapport Allure.
- Collection Postman exécutable par Newman avec rapport HTML et JSON.
- Plan JMeter à 50 utilisateurs avec tableau de bord HTML.
- Scan ZAP limité à Formy et recommandations de réduction du risque.
- Jenkinsfile versionné pour les étapes UI et API.

Éthique et portée : les essais n'utilisent que les cibles de formation nommées dans le sujet, des données fictives et une charge limitée au scénario demandé.

2. Tests UI — Selenium / Allure

Le script `tests/ui/formy.spec.js` utilise Selenium WebDriver, Chrome et Mocha. Il couvre la saisie d'un formulaire, les boutons radio, une liste déroulante, une modale, une alerte JavaScript et un changement d'onglet. Ces mécanismes répondent aux éléments d'interface demandés dans le sujet.

Indicateur	Résultat d'exécution
Tests exécutés	5
Réussis	5
Échoués	0
Durée totale	12,7 s
Rapport	<code>reports/ui/allure/index.html</code>

Une incompatibilité initiale entre Chrome 154 et Chromedriver 152 a été détectée puis corrigée en installant le driver officiel correspondant à la version du navigateur. Cette correction améliore la reproductibilité et évite des résultats faussement négatifs.

3. Tests API — Postman / Newman

La collection `postman/reqres-collection.json` vérifie les opérations CRUD demandées : liste, lecture d'un utilisateur, création, mise à jour et suppression. Les assertions contrôlent les codes HTTP, les champs JSON attendus, les données de test et les horodatages.

Endpoint	Statut attendu	Résultat
GET /api/users?page=2	200	Réussi
GET /api/users/2	200	Réussi
POST /api/users	201	Réussi
PUT /api/users/2	200	Réussi
DELETE /api/users/2	204	Réussi

Bilan : 5 requêtes, 11 assertions et 0 échec. Les rapports détaillés sont `reports/api/newman.html` et `reports/api/newman.json`.

4. Performance — JMeter

Le plan `performance/reqres-50-users.jmx` démarre 50 utilisateurs virtuels sur 10 secondes. Chaque utilisateur effectue une requête GET sur la liste d'utilisateurs, puis une requête POST avec des données de démonstration. Le run produit 100 échantillons au total.

Métrique	Résultat observé
Échantillons	100
Erreurs	3 (3%)
Temps moyen global	29 940 ms
Temps maximal	75 195 ms
Erreurs identifiées	2 réponses HTTP 429 (rate limiting) ; 1 erreur de socket locale

Interprétation : le test répond au seuil demandé de 50 utilisateurs et révèle une dégradation sensible sous concurrence. Les résultats ne permettent pas d'attribuer entièrement la cause au service, car une limite du service tiers et une limite réseau locale sont présentes. Le dashboard JMeter conserve le détail des métriques.

5. Sécurité — OWASP ZAP

Un scan automatisé ZAP a été exécuté uniquement sur `https://formy-project.herokuapp.com/`. Le proxy ZAP a été déplacé sur le port local 8090 pour ne pas entrer en conflit avec Burp sur 8080.

Niveau	Nombre	Exemples
High	0	Aucun
Medium	4	Absence de token anti-CSRF, CSP absente, SRI manquant, bibliothèque JS vulnérable
Low	7	Cookies sans Secure/SameSite, HSTS absent, X-Content-Type-Options absent, timestamp divulgué
Informationnel	5	Commentaires suspects et technologie détectée

Le scan ne confirme pas une exploitation XSS. Les recommandations prioritaires sont : activer CSP et HSTS, mettre à jour la bibliothèque JavaScript signalée, utiliser des tokens anti-CSRF sur les actions sensibles, et appliquer les attributs `Secure`, `HttpOnly` et `SameSite` aux cookies.

6. CI/CD, livrables et conclusion

Le `Jenkinsfile` définit trois étapes : installation reproductible par `npm ci`, tests UI avec génération Allure, puis tests API Newman. Il archive les rapports afin que les preuves d'exécution soient consultables après chaque pipeline. Le plan JMeter et le script ZAP sont disponibles comme étapes déclenchables séparément pour éviter une charge ou un scan externe à chaque commit.

Preuve CI : le job local `qaops-automation-project` a exécuté avec succès le Build #1 en 32,7 secondes : clone Git, installation, Selenium et Newman terminés sans échec.

Livrables versionnés

- Scripts Selenium : `tests/ui/`
- Collection Postman : `postman/reqres-collection.json`
- Plan de tests PDF : `docs/plan-de-tests.pdf`
- Rapports Allure, Newman, JMeter et ZAP : `reports/`
- Rapport final PDF : `docs/rapport-final.pdf`

Conclusion

Le projet fournit une chaîne QAOps reproductible et mesurable. Les validations fonctionnelles UI et API sont toutes réussies. Le test de charge met en évidence un comportement dégradé sous 50 utilisateurs et le scan ZAP identifie des améliorations de configuration de sécurité. La suite peut être enrichie par des tests négatifs, des seuils de performance et une exécution programmée dans Jenkins.